Biblioteca para Manipulação de Grafos — Parte 2: Caminhos Mínimos, Filas de Prioridade e Redes de Colaboração

COS 242 — Teoria dos Grafos (PESC/COPPE/UFRJ)

Arthur Sampaio Carlos Bruno
(Repositório: `TrabalhoGrafos-parte-2`)

Novembro de 2026

Resumo

Este relatório descreve as decisões de projeto, estruturas de dados, otimizações algorítmicas e avaliações experimentais referentes à Parte 2 do trabalho prático da disciplina COS 242 (Teoria dos Grafos). O escopo desta etapa estende a biblioteca em C++17 desenvolvida na Parte 1 para suportar grafos não-direcionados com pesos nas arestas, verificação de pesos negativos e cálculo de distâncias e caminhos mínimos. Quando o grafo não apresenta pesos, a biblioteca emprega busca em largura (BFS) em tempo $\mathcal{O}(V + E)$; na presença de pesos não-negativos, emprega o algoritmo de Dijkstra em duas variantes de fila de prioridade: busca linear em vetor ($\mathcal{O}(V^2 + E)$) e heap binário mínimo indexado ($\mathcal{O}((V + E) \log V)$) com operação `decreaseKey` em $\mathcal{O}(\log V)$. Apresentam-se os resultados empíricos obtidos para a série de grafos ponderados (`grafo_W_3` a `grafo_W_5`), a extrapolação fundamentada de custo quadrático para os grafos de escala extrema (`grafo_W_5` a `grafo_W_7` no modo vetor) e a análise da rede de colaboração científica (DBLP) centrada em Edsger W. Dijkstra.

1 Introdução e Arquitetura da Biblioteca

Na Parte 1 deste projeto, demonstrou-se que a representação por matriz de adjacências (mesmo compactada ao nível de bits) torna-se inviável em memória principal para grafos esparsos com milhões de vértices devido à sua complexidade espacial quadrática $\Theta(V^2)$. Por essa razão, na Parte 2 os módulos legados de matriz de bits (`BitMatrixGraph`) e rotinas genéricas antigas de entrada/saída foram removidos do *build* ativo, consolidando toda a biblioteca sobre a estrutura **CSR (*Compressed Sparse Row*)** implementada na classe `CSRGraph`.

O projeto mantém compilação estrita em C++17 via CMake com otimizações de produção (`-O3 -march=native -DNDEBUG`) para o executável principal `grafo`, enquanto preserva as verificações de corretude (`-UNDEBUG`) ativas no executável de testes unitários `test_grafo`. A suíte de testes valida integralmente em memória a detecção de arestas com peso negativo (como a aresta $(3, 4)$ de peso $-9{,}5$ da Figura 1 do enunciado), a equivalência exata entre o Dijkstra por vetor e por heap, a atualização de caminhos subótimos via `decreaseKey`, componentes desconexas e a leitura de arquivos ponderados.

1.1 Representação CSR Ponderada e Leitura em Duas Passagens

Para representar um grafo não-direcionado ponderado $G = (V, E, w)$ com $n = |V|$ vértices (indexados de 1 a n) e $m = |E|$ arestas, a classe `CSRGraph` adota o padrão *Structure of Arrays* (SoA) em três vetores contíguos:

- `offsets_`: vetor de `std::size_t` de dimensão $n+2$, no qual a vizinhança do vértice u ocupa o intervalo semiaberto $[$`offsets_`$[u], $`offsets_`$[u+1])$;
- `edges_`: vetor de `int` de dimensão $2m$, contendo os vértices adjacentes;
- `weights_`: vetor de `float` de dimensão $2m$, alinhado posição a posição com `edges_`, armazenando os pesos das respectivas arestas.

O consumo exato de memória da estrutura CSR carregada é de $(n+2) \times 8 + 2m \times 4 + 2m \times 4 = 8n + 16m + 16$ bytes. Uma decisão de engenharia fundamental implementada nesta etapa foi a **construção do CSR em duas passagens diretas sobre o arquivo**, eliminando o vetor intermediário de arestas brutas (`temp_edges`) que coexistia com o CSR final na versão inicial:

1. **Primeira passagem:** lê o arquivo sequencialmente para contabilizar o grau de cada vértice, validar o total de arestas válidas (ignorando autoloops e linhas malformadas) e detectar a existência de pesos negativos (`has_negative_weight_`) ou pesos distintos de $1{,}0$ (`is_weighted_`), sem armazenar as arestas em RAM.
2. **Alocação e segunda passagem:** calcula as somas prefixadas em `offsets_`, aloca os vetores finais `edges_` e `weights_` no tamanho exato $2m$, reposiciona o fluxo de leitura no início do arquivo e preenche diretamente as posições definitivas do CSR.

Embora a leitura em duas passagens dobre o trabalho de *parsing* textual — mantendo a complexidade temporal assintótica em $\mathcal{O}(V+E)$ —, ela reduz drasticamente o pico de memória durante o carregamento. Em um grafo com dezenas de milhões de arestas, evita-se a alocação simultânea de $\mathcal{O}(E)$ estruturas temporárias (cerca de 12 bytes extras por aresta de entrada), permitindo carregar instâncias volumosas como `grafo_W_4` ($476{,}84$ MB) e `grafo_W_5` ($648{,}50$ MB) com folga dentro do orçamento de memória física.

2 Decisões de Projeto nos Algoritmos de Caminho Mínimo

A biblioteca unifica o cálculo de distâncias e caminhos mínimos respeitando a natureza do grafo carregado:

- **Grafos sem pesos nas arestas:** utiliza busca em largura (`bfs_shortest_path`) em tempo $\mathcal{O}(V+E)$, garantindo que o menor número de arestas determine a distância.
- **Grafos com pesos negativos:** os algoritmos de Dijkstra verificam a flag `hasNegativeWeight()` em $\mathcal{O}(1)$ antes de iniciar a exploração e lançam imediatamente uma exceção `std::invalid_argume` Isso protege o usuário contra resultados silenciosamente incorretos, visto que a propriedade gulosa do algoritmo de Dijkstra pressupõe subcaminhos monotonicamente não-decrescentes ($w(u,v) \geq 0$).
- **Grafos com pesos não-negativos:** emprega o algoritmo de Dijkstra em duas implementações distintas de gerenciamento de candidatos: **Vetor** e **Heap Binário Indexado**.

Para evitar erros de truncamento por ponto flutuante na soma acumulada de múltiplos pesos `float` ao longo de caminhos longos, o vetor de distâncias acumuladas `dist` utiliza precisão dupla (`double`), enquanto os predecessores são mantidos em um vetor `parent` de inteiros de 32 bits.

2.1 Dijkstra com Vetor (`VectorDistanceStore`)

Na implementação baseada em vetor, mantém-se a distância provisória de todos os vértices em um arranjo linear, acompanhado dos vetores booleanos `visited_` e `queued_`. A cada iteração, a operação `extractMin` percorre linearmente os V vértices em $\mathcal{O}(V)$ para localizar

o vértice não visitado de menor distância acumulada. Uma melhoria importante introduzida no projeto foi o controle estrito de candidatos pendentes por meio do vetor `queued_` e do contador `remaining_active_`: um vértice incrementa o contador apenas na primeira vez em que é descoberto, e o decrementa ao ser extraído. Isso elimina atualizações redundantes em múltiplos relaxamentos de uma mesma aresta e evita uma varredura final $\mathcal{O}(V)$ inútil quando a componente conexa da origem se esgota antes de visitar todo o grafo. Ainda assim, para uma componente conexa de V_r vértices e E_r arestas alcançáveis, a complexidade dominante permanece quadrática:

$$T_{\text{vetor}}(V, E) = \mathcal{O}(V_r \cdot V + E_r) \subseteq \mathcal{O}(V^2 + E).$$

2.2 Dijkstra com Heap Binário Indexado (`IndexedMinHeap`)

Em vez de utilizar `std::priority_queue` da biblioteca padrão do C++ — que não suporta a atualização direta de chaves e exige inserções duplicadas (*lazy deletion*), podendo elevar o tamanho do heap para $\mathcal{O}(E)$ —, implementou-se do zero a classe `IndexedMinHeap`. Essa estrutura mantém no máximo V elementos ativos simultaneamente por meio de três arranjos sincronizados:

- `heap_`: árvore binária completa representada em vetor contendo os identificadores dos vértices ordenados por distância;
- `pos_`: vetor de mapeamento inverso de tamanho $V + 1$, onde `pos_`$[v]$ armazena o índice exato do vértice v dentro de `heap_` (ou -1 se ausente);
- `key_`: vetor com as prioridades (`double`) correntes de cada vértice.

Quando o relaxamento de uma aresta (u, v) reduz a distância estimada de v, o método `pushOrDecreaseKey` consulta `pos_`$[v]$ em $\mathcal{O}(1)$, atualiza a chave e executa `siftUp` em $\mathcal{O}(\log V)$, trocando simultaneamente os elementos em `heap_` e seus índices em `pos_`. Com até V extrações de mínimo ($\mathcal{O}(\log V)$) e até E operações de `siftUp` ($\mathcal{O}(\log V)$), a complexidade total é rigorosamente:

$$T_{\text{heap}}(V, E) = \mathcal{O}((V + E) \log V).$$

2.3 Parada Antecipada para Alvos Específicos (`dijkstraToTargets`)

No Item 1 do estudo de caso, deseja-se obter a distância e o caminho mínimo entre o vértice 10 e apenas cinco vértices de destino: $\{20, 30, 40, 50, 60\}$. Executar o Dijkstra completo até esgotar todos os milhões de vértices do grafo realizaria trabalho desnecessário após a fixação dos cinco alvos. Implementou-se, portanto, a função `dijkstraToTargets`, que monitora os alvos solicitados e interrompe a busca imediatamente quando todos os destinos presentes na mesma componente conexa são extraídos como mínimo da fila de prioridade. Como os pesos são não-negativos, no instante em que um vértice é extraído do heap sua distância já é definitiva. Essa otimização é empregada exclusivamente na geração da tabela do Item 1; no benchmark de desempenho do Item 2, todas as buscas continuam sendo completas (de um vértice para todos os outros), exatamente como exige o enunciado.

3 Estudo de Caso 1: Grafos Ponderados (`grafo_W_3` a `grafo_W_7`)

Os experimentos foram conduzidos sobre os grafos ponderados da série W. A Tabela 1 sintetiza as métricas estruturais, o consumo de memória da representação CSR e o tempo de carregamento em duas passagens para os grafos avaliados experimentalmente (`grafo_W_3`, `grafo_W_4` e `grafo_W_5`).

Tabela 1: Dimensões estruturais, memória CSR e tempo de leitura dos grafos ponderados avaliados.

Arquivo	Vértices (V)	Arestas (E)	Memória CSR (MB)	Tempo de Leitura (s)
`grafo_W_3.txt`	100.000	6.099.941	93,46	3,48
`grafo_W_4.txt`	1.000.000	30.999.978	476,84	17,69
`grafo_W_5.txt`	10.000.000	39.999.995	648,50	31,42

3.1 Item 1: Distâncias e Caminhos Mínimos a partir do Vértice 10

A Tabela 2 apresenta as distâncias mínimas exatas, o número de arestas (saltos) e a sequência completa de vértices que compõem o caminho mínimo entre a origem $s = 10$ e os destinos $t \in \{20, 30, 40, 50, 60\}$ para os grafos `grafo_W_3`, `grafo_W_4` e `grafo_W_5`.

Análise da estrutura dos caminhos: Observa-se claramente na Tabela 2 a propriedade de árvore de caminhos mínimos (*shortest-path tree*): em `grafo_W_3`, os caminhos para 20, 40 e 60 compartilham exatamente o mesmo prefixo inicial (`10 -> 2858 -> 35293 -> 3905 -> 24445`) antes de se ramificarem; de modo análogo, em `grafo_W_4`, os caminhos para $30, 40, 50$ e 60 compartilham o tronco inicial `10 -> 873980 -> 182382`. Além disso, verifica-se que um caminho com menor distância ponderada não corresponde necessariamente ao menor número de arestas: em `grafo_W_3`, o caminho até o vértice 20 possui a menor distância ($0{,}8000$), porém percorre 18 arestas de peso muito reduzido, ao passo que o caminho até o vértice 50 utiliza apenas 9 arestas, mas soma distância superior ($0{,}8700$).

3.2 Item 2: Comparação de Desempenho (Heap vs. Vetor) e Extrapolação

Para avaliar o tempo de execução do algoritmo de Dijkstra calculando a distância de uma origem para todos os demais vértices do grafo, o programa utiliza uma semente determinística para sortear exatamente as mesmas origens tanto para o modo Heap quanto para o modo Vetor. Como para $V > 10.000$ o custo quadrático V^2 de uma única busca no modo vetor já supera o limiar de 10^8 operações de varredura (`max_vector_scan_work = 100000000`), o número efetivo de amostras completas em grafos a partir de 100.000 vértices é ajustado para $k = 1$ (mantendo a busca integral até todos os vértices alcançáveis). A Tabela 3 reúne os tempos reais medidos e as estimativas fundamentadas para os casos em que a execução vetorial torna-se proibitiva.

Interpretação e confirmação da teoria assintótica: Os dados empíricos da Tabela 3 confirmam com precisão notável a diferença teórica entre $\mathcal{O}((V + E) \log V)$ e $\mathcal{O}(V^2 + E)$:

1. **Crescimento quadrático exato no modo Vetor:** Ao aumentar o número de vértices em $10\times$ (de $V = 10^5$ em `W_3` para $V = 10^6$ em `W_4`), a teoria prevê que o termo V^2 cresça exatamente $10^2 = 100\times$. Na prática, o tempo medido saltou de $16{,}8956$ s para $1.681{,}7468$ s, um fator experimental de **99**,**54**$\times$.
2. **Inviabilidade prática do modo Vetor a partir de `W_5`:** Quando V passa de 10^6 (`W_4`) para 10^7 (`W_5` e `W_6`), o fator quadrático impõe uma multiplicação adicional por $100\times$, elevando o tempo de uma única busca de 28 minutos para aproximadamente 168.175 s $\approx$ **46**,**7** horas (≈ 2 dias) por grafo. Para `grafo_W_7` ($V = 5 \times 10^7$, ou $50\times$ os vértices de `W_4`), o fator de escala de $50^2 = 2.500\times$ projeta cerca de **48**,**7** dias para uma única execução vetorial, totalizando mais de 52 dias contínuos de CPU caso a sequência completa fosse aguardada sem interrupção. Caso fossem executadas $k = 100$ buscas no modo vetor, já em `W_4` seriam necessárias $\approx 46{,}7$ horas, e em `W_5` mais de 190 dias.
3. **Escalabilidade casi-linear do Heap Indexado:** Em contrapartida, o `IndexedMinHeap` resolve a busca completa em `grafo_W_3` em apenas $0{,}137$ s, em `grafo_W_4` em $1{,}68$ s e

Tabela 2: Distâncias e caminhos mínimos do vértice 10 para os vértices $20, 30, 40, 50$ e 60.

Grafo	Dest.	Distância	Saltos	Caminho Mínimo ($10 \rightsquigarrow$ Destino)
W_3	20	0,8000	18	10 -> 2858 -> 35293 -> 3905 -> 24445 -> 39345 -> 61279 -> 41321 -> 70540 -> 26049 -> 84450 -> 68904 -> 87862 -> 86778 -> 27694 -> 16608 -> 86355 -> 15760 -> 20
W_3	30	0,8900	14	10 -> 2858 -> 35293 -> 96094 -> 85187 -> 53177 -> 64898 -> 87087 -> 77935 -> 42304 -> 78071 -> 93985 -> 27642 -> 26151 -> 30
W_3	40	0,8400	10	10 -> 2858 -> 35293 -> 3905 -> 24445 -> 86286 -> 14327 -> 19697 -> 15930 -> 46307 -> 40
W_3	50	0,8700	9	10 -> 56663 -> 84437 -> 19040 -> 82510 -> 17622 -> 94992 -> 70948 -> 19878 -> 50
W_3	60	0,9700	16	10 -> 2858 -> 35293 -> 3905 -> 24445 -> 89584 -> 2874 -> 99253 -> 66532 -> 33719 -> 96654 -> 75728 -> 79865 -> 74717 -> 59399 -> 8718 -> 60
W_4	20	2,6600	13	10 -> 532245 -> 877016 -> 554617 -> 430107 -> 437166 -> 805646 -> 841402 -> 663364 -> 655748 -> 634947 -> 318070 -> 807260 -> 20
W_4	30	2,0200	17	10 -> 873980 -> 182382 -> 261305 -> 81769 -> 694525 -> 219466 -> 207765 -> 44636 -> 183747 -> 920911 -> 35338 -> 239369 -> 417194 -> 132461 -> 740528 -> 707581 -> 30
W_4	40	2,5200	15	10 -> 873980 -> 182382 -> 674971 -> 557022 -> 188455 -> 418550 -> 275098 -> 277633 -> 801708 -> 730863 -> 274827 -> 386086 -> 188007 -> 477496 -> 40
W_4	50	2,6900	18	10 -> 873980 -> 182382 -> 261305 -> 81769 -> 694525 -> 382144 -> 748624 -> 663332 -> 149827 -> 349584 -> 735867 -> 9905 -> 716970 -> 472621 -> 407659 -> 994210 -> 630598 -> 50
W_4	60	2,4100	19	10 -> 873980 -> 182382 -> 674971 -> 106317 -> 27985 -> 280367 -> 227336 -> 364950 -> 883474 -> 283156 -> 161217 -> 224210 -> 188021 -> 614364 -> 358029 -> 855615 -> 822462 -> 774634 -> 60
W_5	20	17,3400	13	10 -> 7622850 -> 4615833 -> 6528266 -> 1295458 -> 7103730 -> 8790605 -> 7788327 -> 6766415 -> 7155379 -> 2031914 -> 3908338 -> 8885874 -> 20
W_5	30	17,2400	18	10 -> 2846336 -> 5413988 -> 202621 -> 9270546 -> 6836812 -> 2983314 -> 8775277 -> 5549785 -> 3485759 -> 4341576 -> 8868678 -> 3166991 -> 5117196 -> 7291082 -> 1816279 -> 5664093 -> 8947758 -> 30
W_5	40	17,3700	14	10 -> 1299954 -> 1419794 -> 9885733 -> 5641294 -> 409605 -> 4105715 -> 660832 -> 5850281 -> 3215052 -> 7676193 -> 9008138 -> 4846487 -> 9798152 -> 40
W_5	50	15,4300	12	10 -> 1299954 -> 9927727 -> 8451042 -> 5368621 -> 4276332 -> 3716282 -> 3724752 -> 700154 -> 9271362 -> 6416250 -> 396789 -> 50
W_5	60	17,9900	16	10 -> 8226464 -> 2224619 -> 4275535 -> 1274537 -> 2254373 -> 2345809 -> 4069363 -> 5114103 -> 8764929 -> 6245853 -> 6145878 -> 805658 -> 5288008 -> 7648812 -> 5741162 -> 4688066 -> 60

em `grafo_W_5` (10 milhões de vértices e 40 milhões de arestas) em **24**,**86** s. A vantagem relativa do heap sobre o vetor cresce de $123\times$ em `W_3` para quase $1.000\times$ em `W_4` e cerca de $6.760\times$ em `W_5`, demonstrando que o heap indexado é indispensável para grafos esparsos de larga escala.

Tabela 3: Tempo por busca completa do algoritmo de Dijkstra: valores medidos e estimativas por extrapolação assintótica ($k = 1$ busca completa em grafos com $V \geq 10^5$).

Grafo	Vértices (V)	Arestas (E)	Dijkstra Heap	Dijkstra Vetor	Razão (Vetor
`grafo_W_3.txt`	100.000	6.099.941	0,1372 s (137,2 ms)	16,8956 s ($\approx$ 16,9 s)	
`grafo_W_4.txt`	1.000.000	30.999.978	1,6846 s (1.684,6 ms)	1.681,7468 s ($\approx$ 28,0 min)	
`grafo_W_5.txt`	10.000.000	39.999.995	24,8586 s (24.858,6 ms)	$\approx$ 46,7 horas*	$\approx$
`grafo_W_6.txt`	10.000.000	—	—	$\approx$ 46,7 horas*	
`grafo_W_7.txt`	50.000.000	—	—	$\approx$ 48,7 dias*	

*Valores estimados por extrapolação quadrática a partir do tempo medido em `grafo_W_4` (1.681,75 s para $V = 10^6$).

4 Estudo de Caso 2: Rede de Colaboração Científica (DBLP)

O segundo estudo de caso analisa a rede de coautoria científica extraída da base DBLP (`rede_colaboracao.txt` e `rede_colaboracao_vertices.txt`), composta por $V = 722.385$ pesquisadores e $E = 2.272.543$ relações de coautoria ponderadas. Na modelagem adotada pela instância, o peso de uma aresta entre dois pesquisadores é inversamente proporcional à intensidade de colaboração entre ambos (isto é, $w(u, v) = 1/c_{uv}$, onde c_{uv} é o número de artigos publicados em conjunto); logo, menores pesos indicam colaborações mais fortes e frequentes.

Tomando como origem o pesquisador **Edsger W. Dijkstra** (vértice 2722), executou-se o algoritmo de Dijkstra com heap indexado para determinar a distância e o caminho mínimo até cinco pesquisadores da Ciência da Computação. Os resultados consolidados encontram-se na Tabela 4.

4.1 Interpretação dos Resultados na Rede de Colaboração

A análise dos caminhos mínimos obtidos na rede DBLP revela propriedades estruturais relevantes tanto sob a ótica da Teoria dos Grafos quanto da história da Ciência da Computação:

1. **Inalcançabilidade de Alan M. Turing (distância $= \infty$):** Alan Turing (falecido em 1954) publicou seus trabalhos seminais predominantemente como autor único e antes da consolidação das bases modernas de coautoria indexadas no DBLP. Dessa forma, seu vértice encontra-se isolado ou em uma componente desconexa da componente gigante na qual Edsger W. Dijkstra está inserido, sendo corretamente reportado pela biblioteca com distância infinita.
2. **Papel das pontes de colaboração intensiva:** Todos os quatro pesquisadores alcançáveis partem de Dijkstra por apenas dois ramos iniciais principais:
   - O ramo holandês de métodos formais e algoritmos distribuídos (`Edsger W. Dijkstra -> A. J. M. van Gasteren -> Gerard Tel -> Hans L. Bodlaender`), que conecta Dijkstra à comunidade de Teoria da Computação e Algoritmos em Grafos, conduzindo a **Jon M. Kleinberg** (via Avi Wigderson e Prabhakar Raghavan) e a **Éva Tardos** (via Jan van Leeuwen, Robert E. Tarjan e Andrew V. Goldberg). Notavelmente, A. J. M. van Gasteren foi doutoranda e coautora frequente de Dijkstra, conferindo peso muito baixo à primeira aresta.
   - O ramo de computação científica e sistemas distribuídos (`Edsger W. Dijkstra -> John R. Rice -> Dan C. Marinescu`), que conecta Dijkstra à comunidade de Redes de Computadores e Avaliação de Desempenho, conduzindo tanto a **J. B. Kruskal** (via R. Srikant e Albert G. Greenberg) quanto ao Prof. **Daniel R. Figueiredo** (via Zhi-Li Zhang e Donald F. Towsley, coautor direto e orientador na área de redes).

Tabela 4: Distâncias e caminhos mínimos na rede de colaboração a partir de **Edsger W. Dijkstra** (ID 2722).

Pesquisador Alvo (ID)	Distância	Saltos	Caminho Mínimo de Coautoria
Alan M. Turing (- - -)	∞	—	*Inalcançável (sem caminho na componente conexa de Dijkstra).*
J. B. Kruskal (471365)	$3{,}480368$	8	Edsger W. Dijkstra [2722] -> John R. Rice [9490] -> Dan C. Marinescu [7200] -> Howard Jay Siegel [10343] -> Edwin K. P. Chong [646765] -> Ness B. Shroff [490368] -> R. Srikant [10746] -> Albert G. Greenberg [3655] -> J. B. Kruskal [471365]
Jon M. Kleinberg (5709)	$2{,}706994$	9	Edsger W. Dijkstra [2722] -> A. J. M. van Gasteren [217250] -> Gerard Tel [11456] -> Hans L. Bodlaender [768] -> Dimitrios M. Thilikos [11448] -> Prabhakar Ragde [101826] -> Avi Wigderson [12242] -> Eli Upfal [11834] -> Prabhakar Raghavan [9608] -> Jon M. Kleinberg [5709]
Éva Tardos (11386)	$2{,}753514$	11	Edsger W. Dijkstra [2722] -> A. J. M. van Gasteren [217250] -> Gerard Tel [11456] -> Hans L. Bodlaender [768] -> Jan van Leeuwen [6479] -> Mark H. Overmars [8528] -> Micha Sharir [10572] -> Haim Kaplan [357587] -> Robert Endre Tarjan [11649] -> Andrew V. Goldberg [3694] -> Serge A. Plotkin [318911] -> Éva Tardos [11386]
Daniel R. Figueiredo (343930)	$2{,}942831$	8	Edsger W. Dijkstra [2722] -> John R. Rice [9490] -> Dan C. Marinescu [7200] -> Chuang Lin [391667] -> Bo Li [371226] -> Y. Thomas Hou [4379] -> Zhi-Li Zhang [68773] -> Donald F. Towsley [11466] -> Daniel R. Figueiredo [343930]

3. **Distância ponderada vs. número de saltos:** Assim como observado nos grafos sintéticos, um caminho com mais coautorias intermediárias pode apresentar menor custo total. Por exemplo, o caminho até **Éva Tardos** possui 11 saltos e distância total $2{,}753514$ (média de $\approx 0{,}25$ por aresta, equivalendo a cerca de 4 artigos em conjunto por par de coautores ao longo da cadeia), sendo mais curto na métrica ponderada do que o caminho até **J. B. Kruskal**, que possui apenas 8 saltos, mas distância $3{,}480368$ devido a elos com menor número de artigos coautorados.

5 Conclusão

A extensão da biblioteca para a Parte 2 cumpriu integralmente os requisitos funcionais e de desempenho propostos. Do ponto de vista de memória, a leitura do formato CSR em duas passagens eliminou a sobrecarga de estruturas temporárias $\mathcal{O}(E)$, viabilizando o processamento de grafos com dezenas de milhões de arestas em poucas centenas de megabytes. Do ponto de vista algorítmico, os experimentos demonstraram empiricamente o abismo entre a complexidade quadrática $\mathcal{O}(V^2 + E)$ do Dijkstra com vetor — cujo tempo cresce exatamente $100\times$ a cada aumento de $10\times$ em V, tornando-se impraticável a partir de 10^7 vértices — e a eficiência $\mathcal{O}((V + E) \log V)$ do heap binário mínimo indexado, capaz de solucionar buscas completas em grafos de 10 milhões de vértices em menos de 25 segundos.